

AI활용프로그래밍

Week 5. 함수와 라이브러리

코드에 이름 붙여 다시 쓰기 · 남이 만든 도구 가져다 쓰기

학습 목표

- 자주 쓰는 코드에 이름을 붙여 다시 쓸 수 있다.
- 값을 넘겨 주고 결과를 돌려받을 수 있다.
- return과 print의 차이를 설명할 수 있다.
- 이미 만들어진 도구를 가져다 쓸 수 있다.

오늘의 구성

- 1) 코드에 이름 붙이기
- 2) 값을 넘겨 주고 돌려받기
- 3) return과 print는 다르다
- 4) 이미 만들어진 도구 가져다 쓰기
- 5) With AI — 틀린 코드 고치기 실습 3개

같은 코드를 또 쓰게 될 때

- 인사말을 세 사람에게 하려면 세 번 적어야 합니다.
- 고칠 일이 생기면 세 곳을 다 고쳐야 합니다.
- 한 곳만 고치고 나머지를 잊으면 버그가 됩니다.
- 그래서 한 번 적고 이름을 붙여 다시 씁니다.

코드에 이름 붙이기

```
def greet():  
    print("안녕하세요")  
  
greet()
```

실행 결과
안녕하세요

용어 노트

함수(function) — 코드 묶음에 붙인 이름. 전자레인지의 '해동' 버튼처럼 눌러서 씁니다.

핵심 포인트

- ▶ def 로 만들고, 이름을 적어 부릅니다.
- ▶ 만들기만 하면 아무 일도 일어나지 않습니다.
- ▶ 부르는 줄이 있어야 실행됩니다.
- ▶ 안쪽 줄은 들여쓰기로 묶습니다.

값을 넘겨 주기

```
def greet(name):  
    print(name + "님 안녕하세요")  
  
greet("김민수")
```

실행 결과

김민수님 안녕하세요

용어 노트

매개변수(parameter) — 함수를 만들 때 정해 둔 받을 자리. 택배 받을 주소 칸과 같습니다.

인자(argument) — 함수를 부를 때 실제로 넣는 값. 그 칸에 적어 넣는 실제 주소입니다.

핵심 포인트

- ▶ 괄호 안의 name 이 받을 자리입니다.
- ▶ 부를 때 넣은 값이 그 자리에 들어갑니다.
- ▶ 받을 자리를 만들었으면 값을 꼭 넣어야 합니다.

여러 번 불러 쓰기

```
def greet(name):  
    print(name + "님 안녕하세요")  
  
greet("김민수")  
greet("이영희")
```

실행 결과

김민수님 안녕하세요
이영희님 안녕하세요

핵심 포인트

- ▶ 한 번 만들어 두고 값만 바꿔 부릅니다.
- ▶ 인사말을 고칠 일이 생기면 한 곳만 고칩니다.
- ▶ 이것이 이름을 붙이는 가장 큰 이유입니다.

결과를 돌려받고 싶을 때

- 화면에 보여 주는 것으로 끝나지 않을 때가 있습니다.
- 계산 결과를 받아서 또 계산해야 할 때가 그렇습니다.
- 그럴 때는 return 으로 값을 돌려줍니다.
- return 을 만나면 함수는 거기서 끝납니다.



용어 노트

반환(return) — 함수가 결과값을 돌려주는 일. 자판기가 물건을 내주는 것과 같습니다.

값을 돌려주기 (return)

```
def add(a, b):  
    return a + b  
  
print(add(2, 3))
```

실행 결과

5

핵심 포인트

- ▶ 받을 자리는 여러 개를 둘 수 있습니다.
- ▶ return 이 돌려준 값이 add(2, 3) 자리에 들어갑니다.
- ▶ 돌려받은 값은 다시 계산에 쓸 수 있습니다.

return과 print는 다릅니다

- print 는 사람에게 보여 주기만 합니다.
- return 은 코드에게 값을 건네줍니다.
- 보여 주기만 하면 그 값을 다시 쓸 수 없습니다.
- 이것이 이번 주 가장 흔한 실수입니다.

print만 하면 값을 못 씁니다

```
def add(a, b):  
    print(a + b)  
  
result = add(2, 3)  
print(result)
```

실행 결과

5

None

핵심 포인트

- ▶ 화면에는 5가 보이니 맞은 것 같습니다.
- ▶ 그런데 돌려준 값이 없어 result 는 None 입니다.
- ▶ None 은 '아무 값도 없음'을 뜻합니다.
- ▶ print 를 return 으로 바꿔야 합니다.

미리 정해 둔 값 (기본값)

```
def greet(name, msg="안녕"):
    return msg + ", " + name

print(greet("철수"))
print(greet("영희", "반가워"))
```

실행 결과

안녕, 철수
반가워, 영희

핵심 포인트

- ▶ = 뒤에 적은 값이 기본값입니다.
- ▶ 안 넣으면 기본값이 쓰입니다.
- ▶ 넣으면 넣은 값이 기본값을 대신합니다.
- ▶ 자주 쓰는 값을 기본값으로 두면 편합니다.

함수 안의 이름은 밖에서 못 씁니다

- 함수 안에서 만든 이름은 그 안에서만 삽니다.
- 함수가 끝나면 그 이름은 사라집니다.
- 밖에서 부르면 '그런 이름이 없다'고 합니다.
- 밖에서도 쓰려면 return 으로 돌려받아야 합니다.

밖에서 부르면 오류가 납니다

```
def f():  
    x = 3  
  
print(x)
```

실행 결과

```
NameError: name 'x' is not defined
```

핵심 포인트

- ▶ x 는 함수 안에서만 있던 이름입니다.
- ▶ 함수가 끝나면서 같이 사라졌습니다.
- ▶ return x 로 돌려받으면 쓸 수 있습니다.

남이 만든 도구 가져다 쓰기

- 자주 쓰는 기능은 이미 누가 만들어 두었습니다.
- import 로 가져와서 이름 앞에 붙여 씁니다.
- 코랩에는 웬만한 것이 이미 들어 있습니다.
- 없다고 하면 !pip install 이름 한 줄이면 됩니다.



용어 노트

라이브러리(library) — 미리 만들어 놓은 기능 묶음. 도구함에서 필요한 공구를 꺼내 쓰는 것과 같습니다.

모듈(module) — 라이브러리를 이루는 파일 하나. math, random 이 각각 모듈입니다.

math로 제곱근 구하기

```
import math

print(math.sqrt(16))
```

실행 결과

4.0

핵심 포인트

- ▶ 맨 위에 import 를 한 줄 씩입니다.
- ▶ math.sqrt 처럼 앞에 이름을 붙여 부릅니다.
- ▶ sqrt 는 제곱근을 구해 돌려줍니다.
- ▶ 4가 아니라 4.0 인 것에 주의하세요.

random으로 주사위 굴리기

```
import random

print(random.randint(1, 6))

# 실행할 때마다
# 다른 숫자가 나옵니다.
# 1, 6 둘 다 나올 수 있습니다.
```

핵심 포인트

- ▶ randint(1, 6)은 1부터 6 사이 정수를 줍니다.
- ▶ range 와 달리 끝 숫자 6도 나옵니다.
- ▶ 실행할 때마다 결과가 달라집니다.
- ▶ 그래서 이 슬라이드에는 결과를 적지 않았습니다.

이번 주 흔한 실수 다섯 가지

- 1) 만들기만 하고 부르지 않음
- 2) 부를 때 값을 빠뜨림
- 3) return 대신 print 만 씀
- 4) 함수 안에서 만든 이름을 밖에서 씀
- 5) import 를 빠뜨리고 `math.sqrt` 를 씀

With AI: 오늘은 '고치는 용도'로만

- 이번 학기 전반부(1~7주) 규칙입니다.
- '함수를 만들어 줘'라고 하지 않습니다.
- 내가 만든 함수의 틀린 곳만 물어봅니다.
- 함수는 직접 짜 봐야 구조가 손에 붙습니다.



용어 노트

프롬프트(prompt) — AI에게 주는 질문이나 지시. 자세할수록 답이 좋아집니다.

프롬프트 템플릿 (오류 수정)

나는 파이썬을 처음 배웁니다.
아래 함수가 원하는 대로
동작하지 않습니다.

[내 코드]
(함수 정의와 부르는 줄을
함께 붙여넣기)

[기대한 답]
[실제 나온 답 또는 오류]

- 1) 왜 그런지 두 줄로 설명
- 2) 고칠 곳만 표시
- 3) 전체 코드는 주지 마세요

핵심 포인트

- ▶ 함수만 붙이면 안 됩니다. 부르는 줄도 같이 붙입니다.
- ▶ 오류는 대개 부르는 쪽에서 납니다.
- ▶ 기대한 답과 실제 답을 나란히 적습니다.
- ▶ 답을 받으면 내가 직접 고쳐 실행합니다.

실습 1. 값을 빠뜨리고 부른 함수

해야 할 것

- ▶ 아래 코드를 코드 칸에 넣고 실행합니다.
- ▶ 빨간 글씨의 마지막 줄을 읽습니다.
- ▶ 무엇이 빠졌다고 하는지 확인합니다.
- ▶ 고친 뒤 다시 실행합니다.

✓ 체크

실행하면 숫자 하나가 나옵니다.

실행 코드

```
def add(a, b):  
    return a + b
```

```
print(add(1))
```

[이런 오류가 납니다]

```
TypeError: add() missing  
1 required positional  
argument: 'b'
```

[힌트]

missing 은 "빠졌다",
argument 는 "넣는 값"
입니다.

실습 2. return을 빠뜨린 함수

해야 할 것

- ▶ 코드를 실행합니다. 먼저 5가 보입니다.
- ▶ 그런데 그 아래에서 오류가 납니다.
- ▶ NoneType 이 왜 나왔는지 생각합니다.
- ▶ 한 단어만 고쳐 다시 실행합니다.

✓ 체크

실행하면 5와 10이 차례로 나옵니다.

실행 코드

```
def add(a, b):  
    print(a + b)
```

```
result = add(2, 3)  
print(result * 2)
```

[이런 오류가 납니다]
TypeError: unsupported
operand type(s) for *:
'NoneType' and 'int'

[힌트]
화면에 보이는 것과
돌려주는 것은 다릅니다.

실습 3. 함수 안 이름을 밖에서 쓴 코드

해야 할 것

- ▶ 코드를 실행해 오류 이름을 확인합니다.
- ▶ total 이 어디서 만들어졌는지 봅니다.
- ▶ 밖에서도 쓰려면 무엇이 필요할지 생각합니다.
- ▶ 고친 뒤 다시 실행합니다.

✓ 체크

실행하면 3 이 나옵니다.

실행 코드

```
def calc():  
    total = 1 + 2
```

```
calc()  
print(total)
```

```
[ 이런 오류가 납니다 ]  
NameError: name 'total'  
is not defined
```

```
[ 힌트 ]  
return 으로 돌려주고  
받아 두어야 합니다.
```

AI 답변 확인하는 법

- 고친 코드를 직접 실행해 봤는가?
- 값을 바꿔 불러도 잘 되는가?
- 돌려받은 값을 실제로 써 봤는가?
- 고친 이유를 내 말로 말할 수 있는가?

미니 퀴즈 (5문항)

- 1) 만들기만 하고 안 부르면 어떻게 될까요?
- 2) print와 return의 차이는 무엇일까요?
- 3) 돌려주지 않는 함수의 결과는? (0 / None)
- 4) 함수 안에서 만든 이름을 밖에서 쓸 수 있을까요?
- 5) math를 쓰려면 맨 위에 무엇을 적을까요?

정답: 1-아무 일도 안 남, 2-보여주기·돌려주기, 3-None, 4-없음,
5-import math

과제

- 필수 — 섭씨를 화씨로 바꿔 돌려주는 함수 만들기.
- 화씨 = 섭씨 $\times 9 / 5 + 32$ 입니다.
- 0, 25, 100 을 넣어 결과를 적어 둡니다.
- 제출은 코랩 공유 링크 또는 .ipynb 파일.
- 도전(선택) — random으로 가위바위보 내기 정하기.

오늘 정리 & 다음 주

- 자주 쓰는 코드는 이름을 붙여 다시 씁니다.
- 받을 자리를 만들고, 부를 때 값을 넣습니다.
- 돌려받으려면 return 이 필요합니다.
- 이미 만들어진 도구는 import 로 가져옵니다.
- 다음 주 — 값 여러 개를 한 줄에 담아 다룹니다.